

POLITECHNIKA ŚWIĘTOKRZYSKA
Wydział Elektrotechniki, Automatyki i Informatyki

Projekt: Technologie obiektowe	
Temat: Object Document Mapper (ODM)	
Autor: Dominika Jedynak Kacper Kaczmarczyk	Grupa: 1ID21A
	Data wykonania ćwiczenia: 23.06.2026

1. Cel projektu

Celem projektu było stworzenie własnego mechanizmu typu Object Document Mapper (ODM), umożliwiającego dwukierunkową konwersję pomiędzy obiektami języka Java a dokumentami JSON.

Projekt został zrealizowany z wykorzystaniem mechanizmu refleksji (Reflection API), który pozwala na dynamiczny odczyt i modyfikację pól obiektów podczas działania programu. Dodatkowo zaimplementowano własny parser JSON oraz zestaw adnotacji umożliwiających kontrolowanie procesu mapowania.

Aplikacja została wykonana jako program konsolowy.

2. Najważniejsze klasy

JsonOdmMapper

JsonOdmMapper jest główną klasą systemu i pełni rolę wzorca projektowego **Facade**.

Udostępnia prosty interfejs do serializacji i deserializacji obiektów, ukrywając szczegóły działania pozostałych komponentów.

Przykładowe użycie:

```
JsonOdmMapper mapper = new JsonOdmMapper();

String json = mapper.toJson(person);

Person loadedPerson =
    mapper.fromFile(path, Person.class);
```

Klasa wykorzystuje wewnętrznie komponenty odpowiedzialne za serializację, deserializację oraz analizę JSON.

JsonSerializer

Klasa odpowiedzialna za serializację obiektów Java do formatu JSON.

Podczas działania:

- pobiera pola obiektu przy pomocy refleksji,
- odczytuje ich wartości,
- obsługuje typy proste,
- obsługuje obiekty zagnieżdżone,

- obsługuje kolekcje,
- uwzględnia adnotacje sterujące procesem mapowania.

Dzięki temu możliwe jest automatyczne wygenerowanie dokumentu JSON dla dowolnego obiektu spełniającego założenia projektu.

JsonDeserializer

Klasa odpowiedzialna za deserializację dokumentów JSON do obiektów Java.

Podczas działania:

- tworzy nową instancję klasy,
- odczytuje dane zwrócone przez parser,
- przypisuje wartości do pól obiektu,
- odtwarza obiekty zagnieżdżone,
- odtwarza kolekcje.

Proces realizowany jest przy pomocy Reflection API.

JsonParser

JsonParser jest własną implementacją parsera JSON stworzoną na potrzeby projektu.

Parser odpowiada za:

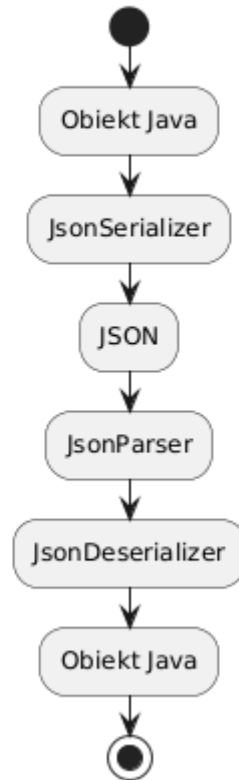
- analizę składni dokumentu JSON,
- rozpoznawanie obiektów JSON,
- rozpoznawanie tablic JSON,
- odczyt wartości prostych,
- podstawową walidację danych wejściowych.

Dzięki własnemu parserowi projekt nie wykorzystuje zewnętrznych bibliotek do mapowania danych.

3. Działanie programu

Schemat działania aplikacji przedstawiono poniżej.

Proces serializacji i deserializacji ODM



Podczas serializacji pola obiektu są odczytywane przy pomocy refleksji i zapisywane do dokumentu JSON.

Podczas deserializacji parser analizuje zawartość dokumentu JSON, a następnie tworzony jest nowy obiekt, którego pola są uzupełniane odpowiednimi wartościami.

Projekt obsługuje:

- String
- int
- long
- double
- boolean
- List
- obiekty zagnieżdżone

4. Adnotacje

W projekcie zastosowano własne adnotacje pozwalające kontrolować proces mapowania.

@JsonName

Pozwala zmienić nazwę pola zapisywanego w dokumencie JSON.

Przykład:

```
@JsonName("first_name")  
private String firstName;
```

@JsonIgnore

Powoduje pominięcie pola podczas serializacji.

Przykład:

```
@JsonIgnore  
private String password;
```

@JsonSkipNull

Powoduje pominięcie pól zawierających wartość null.

@JsonDefaultValue

Pozwala określić wartość domyślną wykorzystywaną podczas deserializacji.

4. Instrukcja uruchomienia

Do uruchomienia projektu wymagana jest Java 21.

Kompilacja i uruchomienie aplikacji:

Linux / macOS

```
./mvnw clean compile exec:java
```

Windows

```
mvnw.cmd clean compile exec:java
```

Aktualizacja zależności i budowanie projektu:

Linux / macOS

```
./mvnw clean install -U
```

Windows

```
mvnw.cmd clean install -U
```

Uruchomienie testów:

Linux / macOS

```
./mvnw test
```

Windows

```
mvnw.cmd test
```

Uruchomienie w kontenerze Docker

```
docker compose run --rm json-odm
```

5. Instrukcja obsługi

Po uruchomieniu programu użytkownik może wybrać jedną z dostępnych opcji:

- zapis przykładowego obiektu klasy Person do formatu JSON,
- odczyt obiektu klasy Person z pliku JSON,
- zakończenie działania programu.

Po wybraniu opcji „1 - **Zapisz obiekt Person do JSON**” program tworzy przykładowy obiekt klasy Person, dokonuje jego serializacji do formatu JSON, a następnie zapisuje wynik do pliku person.json. W konsoli wyświetlany jest obiekt Java oraz wygenerowany dokument JSON.

Następnie po wybraniu opcji „**2 - Odczytaj obiekt Person z JSON**” program odczytuje dane zapisane w pliku person.json i tworzy na ich podstawie nowy obiekt klasy Person. Odtworzony obiekt zostaje wyświetlony w konsoli, co pozwala zweryfikować poprawność procesu deserializacji.

```
=====
1 - Zapisz obiekt Person do JSON
2 - Odczytaj obiekt Person z JSON
q - Wyjście
Wybierz opcję: 1

Obiekt Java:
Person(name=Jan, lastName=Kowalski, age=25, active=true, averageGrade=3.5, address=Address(city=Warszawa, street=Marszałkowska, buildingNumber=10), skills=[Java, SQL], luckyNumbers=[7, 14],
flags=[true, false], grades=[4.0, 4.5], previousAddresses=[Address(city=Warszawa, street=Marszałkowska, buildingNumber=10)], nestedSkills=[[Java, Spring], [Docke]], nullElements=[A, null,
B], emptyList=[], nullableList=null, nick=Jan123, password=haslo123, email=null, country=Polka)

JSON:
{"name":"Jan","lastName":"Kowalski","age":25,"active":true,"averageGrade":3.5,"address":{"city":"Warszawa","street":"Marszałkowska","buildingNumber":10},"skills":["Java","SQL"],"luckyNumbers":
[7,14],"flags":[true,false],"grades":[4.0,4.5],"previousAddresses":[{"city":"Warszawa","street":"Marszałkowska","buildingNumber":10}],"nestedSkills":[["Java","Spring"],["Docke"]],"nullE
lements":["A",null,"B"],"emptyList":[],"nullableList":null,"pseudonim":"Jan123","country":"Polka"}

Zapisano do pliku

=====
1 - Zapisz obiekt Person do JSON
2 - Odczytaj obiekt Person z JSON
q - Wyjście
Wybierz opcję: 2

Odczytany obiekt:
Person(name=Jan, lastName=Kowalski, age=25, active=true, averageGrade=3.5, address=Address(city=Warszawa, street=Marszałkowska, buildingNumber=10), skills=[Java, SQL], luckyNumbers=[7, 14],
flags=[true, false], grades=[4.0, 4.5], previousAddresses=[Address(city=Warszawa, street=Marszałkowska, buildingNumber=10)], nestedSkills=[[Java, Spring], [Docke]], nullElements=[A, null,
B], emptyList=[], nullableList=null, nick=Jan123, password=null, email=null, country=Polka)
```

6. Wnioski

W ramach projektu zaimplementowano własny mechanizm Object Document Mapper (ODM) umożliwiający serializację i deserializację obiektów Java do formatu JSON.

Projekt wykorzystuje Reflection API do dynamicznego odczytu i modyfikacji pól obiektów, własny parser JSON oraz własne adnotacje sterujące procesem mapowania.

Dodatkowo zastosowano wzorzec projektowy Facade w klasie JsonOdmMapper, dzięki czemu użytkownik korzysta z prostego interfejsu ukrywającego szczegóły implementacyjne.

Opracowane rozwiązanie realizuje wszystkie założenia projektowe, obsługując typy proste, kolekcje, obiekty zagnieżdżone oraz podstawową konfigurację procesu mapowania.